

교과목 3 : 초거대언어모델(LLM)

단원 3 : LLM

DAY3. LLM의 주요 기능과 활용 사례

목 차

1. LLM 활용 예제		3
2. 인공지능 챗봇		9
3. AI 음악 추천 챗봇		16

4. LLM 활용 예제

1. LLM 주요 기능

LLM은 자연어를 이해하고 생성하는 AI 모델입니다. 대표 기능은 문장 생성, 질의응답, 요약, 번역, 채팅, 코드 생성, 문서 분석, 고객 상담 자동화, 교육용 튜터, 데이터 분석 보조 등입니다. OpenAI API는 텍스트 생성, 자연어 처리, 이미지 이해 등 다양한 작업을 API로 사용할 수 있습니다. ([OpenAI 개발자](#))

2. 설치와 API Key 설정

```
pip install openai
```

Windows PowerShell:

```
setx OPENAI_API_KEY "발급받은_API_KEY"
```

OpenAI SDK는 환경변수 OPENAI_API_KEY를 자동으로 읽습니다. ([OpenAI 개발자](#))

3. 공통 기본 코드

```
from openai import OpenAI

# OpenAI API 클라이언트 생성
# OPENAI_API_KEY 환경변수를 자동으로 사용합니다.
client = OpenAI()

def ask_llm(prompt: str, instructions: str = "You are a helpful assistant.") -> str:
    """
    OpenAI LLM에 요청을 보내고 응답 텍스트를 반환하는 공통 함수입니다.
    """

    response = client.responses.create(
        model="gpt-5.5",
        instructions=instructions,
        input=prompt
```

```
)
```

```
return response.output_text
```

4. 문장 생성 예제

```
prompt = """
인공지능 교육 홍보 문장을 3개 작성하시오.
대상은 파이썬을 처음 배우는 수강생입니다.
"""

result = ask_llm(
    prompt,
    instructions="마케팅 문구를 자연스럽고 설득력 있게 작성하시오."
)

print(result)
```

5. 질의 응답 예제

```
prompt = """
질문: FastAPI는 무엇이고, 언제 사용하는가?
답변은 교육용으로 쉽게 설명하시오.
"""

result = ask_llm(
    prompt,
    instructions="프로그래밍 강사처럼 핵심 개념과 사용 상황을 설명하시오."
)

print(result)
```

6. 요약 예제

```
text = """
LLM은 대규모 텍스트 데이터를 학습하여 문장을 생성하고,
질문에 답하고, 문서를 요약하고, 번역하고, 사용자와 대화할 수 있는
인공지능 언어 모델이다. 최근에는 챗봇, 고객 상담, 문서 자동화,
코드 생성, 교육 보조, 데이터 분석 등 다양한 분야에서 활용된다.
```

```

"""
prompt = f"""
다음 글을 핵심만 3줄로 요약하시오.

{text}
"""
result = ask_llm(
    prompt,
    instructions="불필요한 설명 없이 핵심만 요약하시오."
)
print(result)

```

7. 번역 예제

```

prompt = """
다음 한국어 문장을 영어로 자연스럽게 번역하시오.
문장:
LLM은 업무 자동화와 교육용 챗봇 개발에 매우 유용하다.
"""
result = ask_llm(
    prompt,
    instructions="전문적이고 자연스러운 영어 번역을 제공하시오."
)
print(result)

```

8. 채팅 예제

```

messages = """
사용자: LLM이 무엇인가요?
AI: LLM은 대규모 언어 모델입니다.
사용자: 그러면 API로 어떻게 사용할 수 있나요?
"""
result = ask_llm(
    messages,

```

```
instructions="이전 대화 흐름을 참고하여 친절하게 이어서 답변하시오."  
)  
print(result)
```

9. 다양한 Use Case 예제

9-1. 코드 생성

```
prompt = """  
Python으로 리스트에서 짝수만 추출하는 함수를 작성하시오.  
"""  
  
result = ask_llm(prompt, instructions="Python 코드를 정확하게 작성하시오.")  
print(result)
```

9-2. 이메일 작성

```
prompt = """  
수강생에게 과제 제출 기한이 내일까지라는 안내 이메일을 작성하시오.  
정중하고 간단하게 작성하시오.  
"""  
  
result = ask_llm(prompt, instructions="업무용 이메일 형식으로 작성하시오.")  
print(result)
```

9-3. 학습 문제 생성

```
prompt = """  
LLM 개념에 대한 4지선다형 문제 5개를 작성하시오.  
각 문제에는 정답도 포함하시오.  
"""  
  
result = ask_llm(prompt, instructions="교육 평가 문항 형식으로 작성하시오.")  
print(result)
```

10. 하나의 실행 파일 예제

```
from openai import OpenAI

client = OpenAI()

def run_example(title: str, prompt: str, instructions: str):
    print("=" * 60)
    print(title)
    print("=" * 60)
    response = client.responses.create(
        model="gpt-5.5",
        instructions=instructions,
        input=prompt
    )
    print(response.output_text)
    print()

run_example(
    "1. 문장 생성",
    "AI 교육 과정을 홍보하는 문장 3개를 작성하시오.",
    "자연스럽고 설득력 있는 문장을 작성하시오."
)

run_example(
    "2. 질의 응답",
    "LLM과 일반 검색엔진의 차이를 설명하시오.",
    "교육용으로 쉽게 설명하시오."
)

run_example(
    "3. 요약",
    "LLM은 문장 생성, 요약, 번역, 질의응답, 채팅 등에 활용된다. 이 내용을 2줄로 요약하시오.",
```

```
"핵심만 간단히 요약하시오."
)

run_example(
    "4. 번역",
    "다음 문장을 영어로 번역하시오: LLM은 다양한 업무 자동화에 활용된다.",
    "자연스러운 영어로 번역하시오."
)

run_example(
    "5. 채팅",
    "사용자가 'OpenAI API는 어디에 쓰나요?'라고 질문했습니다. 답변하시오.",
    "친절한 챗봇처럼 답변하시오."
)
```


2. 인공지능 챗봇

1. 인공지능 챗봇이란?

(1) 챗봇(Chatbot)의 정의

챗봇(Chatbot)은 사람과 자연어(Natural Language)로 대화하는 프로그램입니다.

사용자가 질문하면
↓
질문을 이해하고
↓
적절한 답변을 생성하여
↓
사용자에게 전달합니다.

초기의 챗봇은

- 미리 준비된 답변
- 키워드 검색
- 규칙 기반(Rule-Based)

방식을 사용했습니다.

그러나 최근 ChatGPT 는

- 대규모 언어모델(LLM)
- 딥러닝
- Transformer
- 자연어처리(NLP)

기술을 이용하여 사람과 거의 비슷한 수준의 대화를 수행합니다.

2. OpenAI ChatGPT 란?

OpenAI 에서 개발한 생성형 AI 입니다.

GPT 는 Generative Pre-trained Transformer 의 약자입니다.

즉,

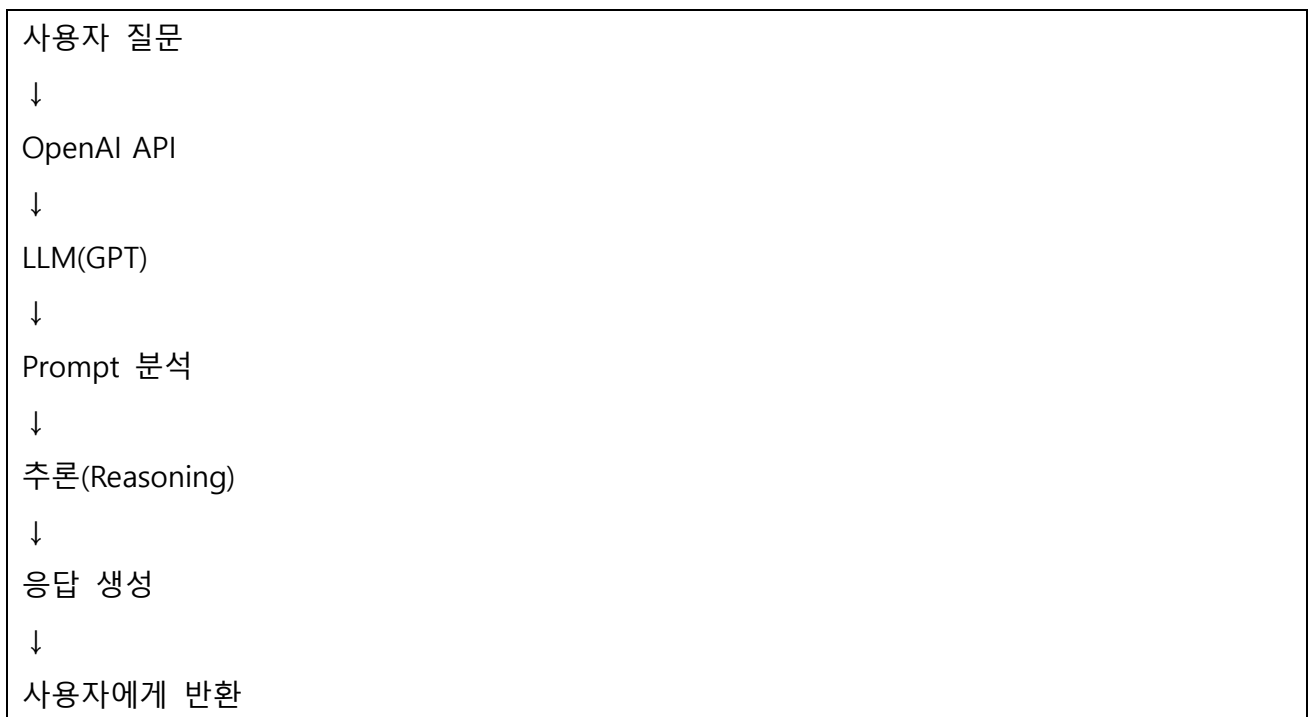
- 대규모 데이터를 미리 학습하고
- 새로운 문장을 생성하는 AI

입니다.

기존 챗봇과 가장 큰 차이는

기존 챗봇	ChatGPT
정해진 답변	새로운 답변 생성
키워드 검색	문맥 이해
규칙 기반	LLM 기반
단순 FAQ	자유로운 대화

3. ChatGPT 의 동작 과정

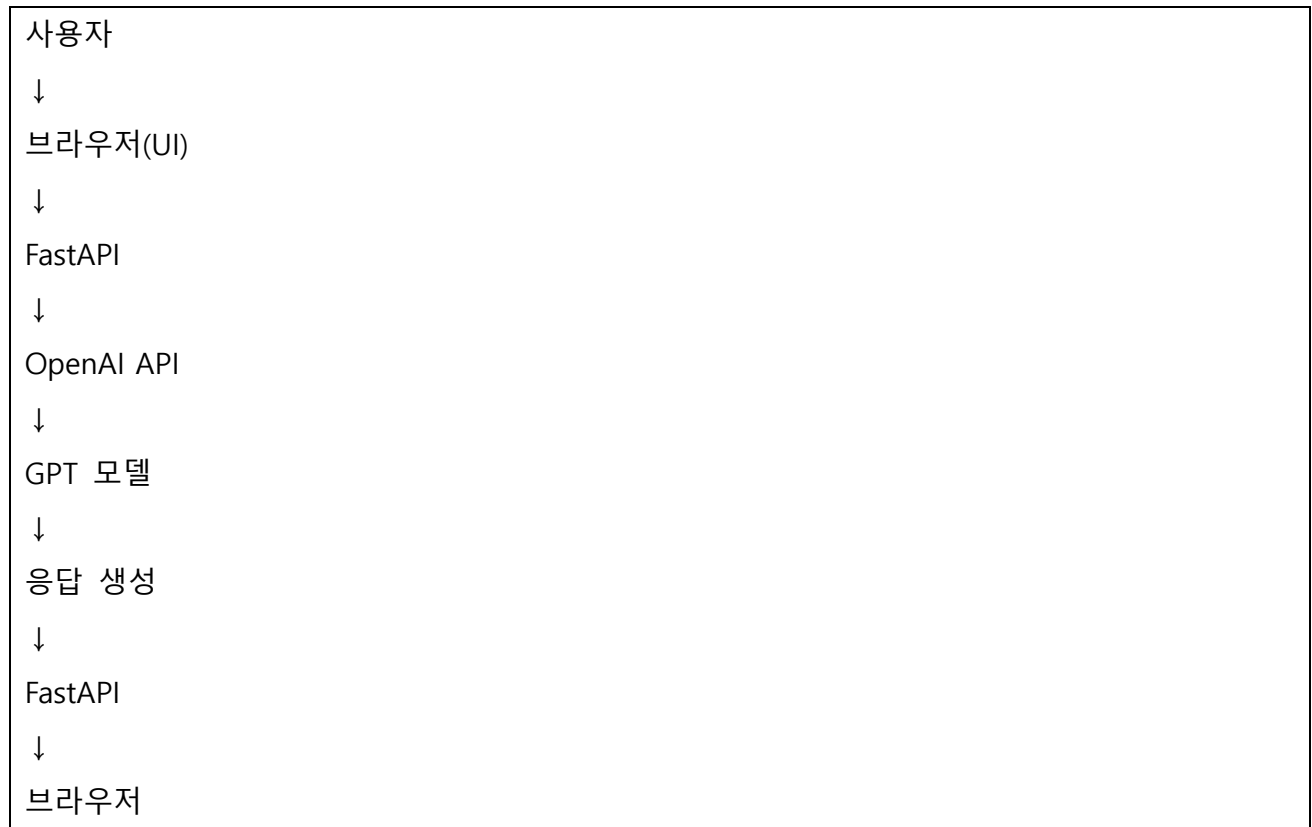


예)



4. ChatGPT 시스템 구성

OpenAI 기반 챗봇은 다음과 같이 구성됩니다.



5. ChatGPT 활용 분야

고객 상담

- FAQ
- 민원 처리
- 주문 조회

교육

- AI 튜터
- 문제 생성
- 코드 설명
- 시험 문제 생성

개발

- 코드 생성
- 코드 리뷰
- 버그 수정

번역

- 한국어 ↔ 영어
- 다국어 지원

문서 자동화

- 계약서 요약
- 회의록 작성
- 이메일 생성

데이터 분석

- SQL 생성
- Python 코드 작성
- Pandas 분석

기업 업무

- 보고서 작성
- 업무 자동화
- 사내 챗봇

6. OpenAI API 개발환경 구축

이는 Python 개발환경을 구축하기 위한 과정입니다.

ChatGPT 개발에서는 다음과 같이 준비하는 것이 일반적입니다.

① Python 설치

권장

Python 3.11

② 가상환경 생성

```
python -m venv .venv
```

③ 가상환경 활성화

Windows

```
.venv\Scripts\activate
```

④ 패키지 설치

```
pip install openai
```

```
pip install fastapi
```

```
pip install uvicorn
```

```
pip install python-dotenv
```

⑤ API Key 발급

OpenAI

↓

API Keys

↓

새 Key 생성

↓

.env 저장

```
OPENAI_API_KEY=sk-xxxxxxx
```

7. Jupyter Notebook 활용

ChatGPT 학습에서도 Notebook 은 다음과 같은 용도로 매우 유용합니다.

- Prompt 실험
- OpenAI API 테스트
- 데이터 전처리
- 모델 성능 비교
- 코드 실습
- 결과 시각화

8. Prompt Engineering

ChatGPT 개발에서 가장 중요한 요소입니다.

Prompt 는 AI 에게 주는 작업 지시문입니다.

예)

당신은 Python 강사입니다.
FastAPI 를 초보자도 이해하도록 설명하세요.

Prompt 가 좋아질수록

- 답변 품질
- 정확도
- 일관성

이 향상됩니다.

9. OpenAI ChatGPT 개발 절차

사용자 요구사항 분석

↓

Prompt 설계

↓

OpenAI API 호출

↓

GPT 응답 생성

↓

FastAPI 에서 반환

↓

브라우저 출력

↓

사용자 피드백

↓

Prompt 개선

10. OpenAI ChatGPT 프로젝트 예시

실습용 프로젝트를 예로 들면 다음과 같은 구조를 구성할 수 있습니다.

```
chatgpt_project/
|
|—— app/
|   |—— main.py
|   |—— routers/
|   |—— services/
|   |—— schemas/
|   |—— config.py
|   |—— security.py
|   |—— templates/
|
|—— static/
|—— .env
|—— requirements.txt
|—— README.md
```

3. AI 음악 추천 챗봇

이 프로젝트는 사용자의 감정(행복, 슬픔, 그리움, 위로, 화남 등)에 맞는 음악을 추천해 주는 **AI 음악 추천 챗봇**입니다.

1. 프로젝트 목표

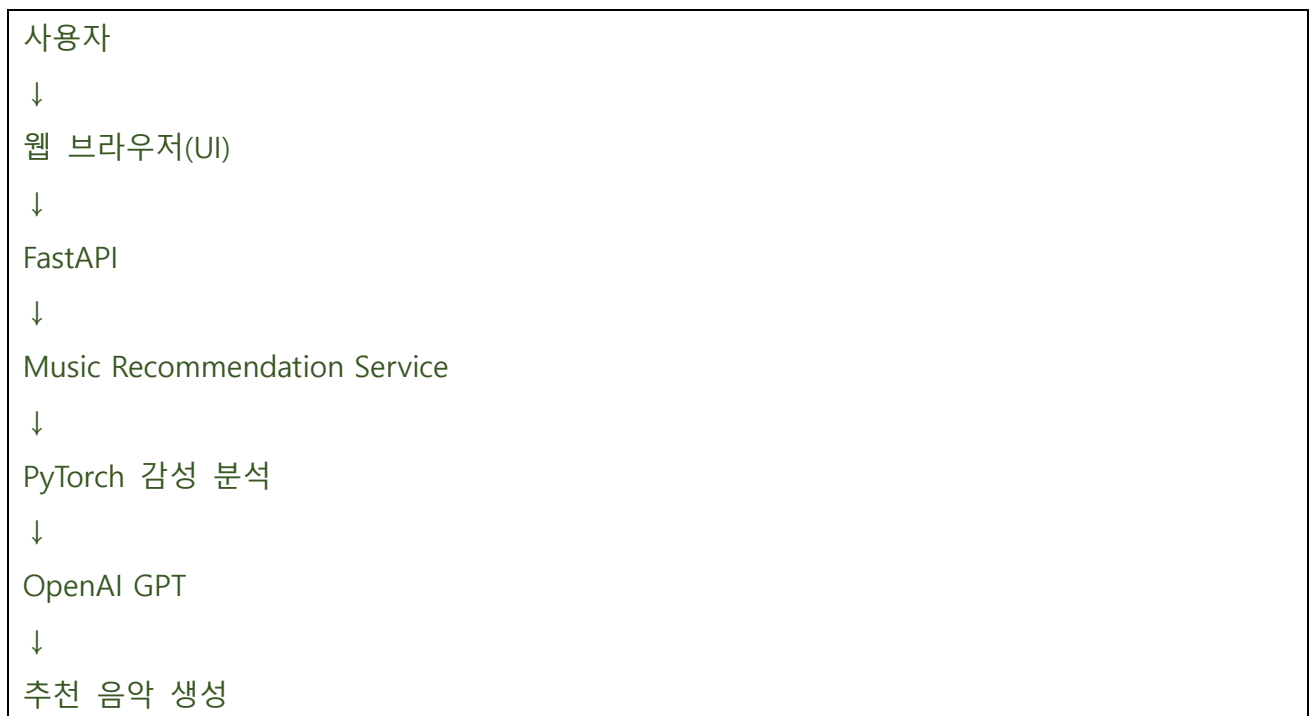
프로젝트의 목표는 다음과 같습니다.

- OpenAI ChatGPT API를 이용한 자연어 기반 음악 추천
- 감성(기분) 분석을 통한 음악 추천
- FastAPI 기반 REST API 구현
- PyTorch를 이용한 감성 벡터 처리
- 음악 추천 데이터 관리
- 추천 결과를 웹 UI에서 확인

즉, 사용자가 자신의 현재 기분을 입력하면 AI가 상황을 이해하고 적절한 음악을 추천하는 챗봇을 만드는 것이 목적입니다.

2. 시스템 구성

전체 구조는 다음과 같습니다.



↓

추천 결과 출력

FastAPI는 전체 프로그램을 제어하고, PyTorch는 감성 벡터 계산을 담당하며, OpenAI는 자연스러운 추천 문장을 생성합니다.

3. 프로젝트 구조

프로젝트는 일반적인 FastAPI 애플리케이션 구조를 따릅니다.

```
openai_music_recommend_chatbot/
|
|—— app/
|   |—— main.py
|   |—— config.py
|   |—— routers/
|   |—— services/
|   |—— schemas/
|   |—— models/
|   |—— data/
|   |—— templates/
|   |—— static/
|   └── utils/
|
|—— music_dataset/
|
|—— requirements.txt
|—— README.md
└── .env
```

각 디렉터리는 역할별로 분리되어 있어 유지보수가 쉽습니다.

4. 주요 기능

① 감성 분석

사용자가 입력한 문장을 분석합니다.

예)

오늘 너무 우울해

↓

감정

슬픔

PyTorch Tensor를 이용하여 감성 점수를 계산합니다.

② 음악 추천

감정이 결정되면 음악 추천 Prompt를 생성합니다.

예)

사용자는 슬픈 상태입니다.

위로가 되는 음악 5곡을 추천하세요.

↓

OpenAI API 호출

↓

GPT가 추천 생성

예)

아이유 - 밤편지

김동률 - 감사

성시경 - 거리에서

③ 설명 생성

단순히 노래 제목만 추천하지 않습니다.

GPT가 왜 이 음악을 추천하는지 까지 설명합니다.

예)

이 노래는 잔잔한 피아노와 따뜻한 가사가 우울한 기분을 안정시켜 줄 수 있습니다.

④ YouTube 검색 URL

감정 + 음악 검색 방법을 사용합니다.

예)

행복할 때 듣는 음악

슬플 때 듣는 음악

그리울 때 듣는 노래

검색하여

YouTube URL도 함께 제공합니다.

⑤ 대화형 챗봇

일반 추천이 아니라 계속 이어지는 대화를 지원합니다.

예)

사용자

슬픈 노래 추천해줘

↓

AI

잔잔한 발라드를 좋아하시나요?

↓

사용자

네

↓

AI

그렇다면...

처럼 이전 대화를 기억합니다.

5. OpenAI API 사용

최신 Responses API를 사용합니다.

```
client.responses.create(...)
```

Prompt에는

- 현재 감정
- 좋아하는 장르
- 추천 개수
- 설명 포함 여부
- YouTube 검색 키워드

등이 전달됩니다.

GPT는

- 음악 추천
- 추천 이유
- 비슷한 노래
- 플레이리스트

까지 생성합니다.

6. PyTorch 사용 이유

이 프로젝트에서는 PyTorch를 이용하여 감정을 벡터(Tensor)로 표현합니다.

예)

```
행복
↓
Tensor
[0.91,
 0.15,
 0.03,
 0.20]
슬픔
↓
[0.04,
 0.89,
 0.12,
 0.07]
```

이 벡터를 이용하면 향후

- BERT
- KoBERT
- RoBERTa
- SentenceTransformer

등으로 쉽게 확장할 수 있습니다.

7. 추천 알고리즘

추천 과정은 다음과 같습니다.

사용자 입력

↓

감정 추출

↓

PyTorch 벡터화

↓

유사 감정 계산

↓

추천 후보 선택

↓

OpenAI Prompt 생성

↓

GPT 추천

↓

결과 출력

8. FastAPI API 구성

예를 들어 다음과 같은 API를 제공합니다.

Method	URL	설명
GET	/	메인 화면
POST	/api/chat	AI 음악 추천
POST	/api/recommend	음악 추천
GET	/api/history	추천 기록
GET	/api/music/list	음악 목록
GET	/api/health	서버 상태

Swagger에서도 테스트 가능합니다.

<http://127.0.0.1:8000/docs>

9. 웹 UI 구성

브라우저에서는 다음과 같은 화면을 제공합니다.

 AI 음악 추천 챗봇

현재 기분

장르

[발라드 ▼]

추천 개수

[5]

[추천받기]

AI 추천 결과

①

②

③

추천 이유

YouTube 검색 링크

10. 확장 가능한 기능

현재 구조는 다음과 같은 기능을 추가하기 쉽도록 설계되어 있습니다.

- Spotify API 연동
- YouTube Data API를 이용한 실제 영상 검색
- 음원 스트리밍 서비스 연동
- 사용자별 추천 이력 저장(MySQL)
- 로그인 및 회원 관리
- 즐겨찾기(Playlist) 기능
- KoBERT 기반 감정 분류
- 벡터 데이터베이스(RAG) 기반 음악 추천
- 실시간 음성 입력(STT) 및 음성 응답(TTS)
- 멀티모달 입력(텍스트 + 이미지 기반 감정 분석)